

DSA Visualizer Platform

Complete Development Roadmap (Based on Taskintern PRD)

Project Overview

Objective

Build an interactive web application that allows students to learn Data Structures and Algorithms through real-time visualization, animations, and hands-on interaction.

The platform should not only visualize algorithms but also teach the concepts behind them through explanations, complexity analysis, and interactive controls.

Recommended Tech Stack

Frontend

- Next.js 15
- TypeScript
- Tailwind CSS
- Framer Motion
- Zustand (State Management)

Visualization

- SVG (Recommended)
- React Flow (For Trees & Graphs)
- D3.js (Optional)

Deployment

- Vercel
-

Suggested Folder Structure

```
src/  
|  
├── app/  
|
```

```
├── components/
│   ├── layout/
│   ├── visualizer/
│   ├── controls/
│   ├── animations/
│   ├── explanation/
│   ├── complexity/
│   └── ui/
├── algorithms/
│   ├── arrays/
│   ├── linkedList/
│   ├── stack/
│   ├── queue/
│   ├── tree/
│   ├── bst/
│   ├── heap/
│   ├── graph/
│   ├── sorting/
│   └── searching/
├── hooks/
├── lib/
├── utils/
├── types/
└── constants/
```

Phase 1 — Project Setup

Goal

Create the project foundation.

Tasks

- Initialize Next.js project
- Configure TypeScript
- Install Tailwind CSS
- Configure ESLint
- Configure Prettier
- Install Framer Motion
- Install Zustand
- Configure folder structure
- Create reusable UI components

Deliverables

- Working project
 - Clean architecture
 - GitHub Repository
-

Phase 2 — Landing Page

Goal

Create an attractive homepage.

Sections

Hero Section

- Project title
- Description
- CTA Button

Features

- Interactive Learning
- Live Visualization
- Complexity Analysis
- Interview Preparation

Supported Data Structures

Supported Algorithms

Technology Used

Footer

Nice Animations

- Floating shapes
 - Gradient background
 - Hover cards
 - Scroll animations
-

Phase 3 — Dashboard Layout

Goal

Create the learning workspace.

Sidebar

Data Structures

Arrays

Linked List

Stack

Queue

Tree

BST

Heap

Graph

Algorithms

Sorting

Searching

Graph Algorithms

Main Area

Controls

Visualization Canvas

Explanation Panel

Complexity Table

Deliverables

- Responsive dashboard
- Sidebar navigation
- Layout system

Phase 4 — Array Visualizer

Goal

Implement complete Array visualization.

Operations

- Insert
- Delete
- Update
- Search
- Traverse
- Reset

Visualization

```
[10][25][7][90][55]
```

Animations

- Insert animation
- Delete animation
- Highlight searched item
- Traversal animation

Learning Panel

Concept

Working

Example

Applications

Advantages

Disadvantages

Complexity

Phase 5 — Linked List Visualizer

Operations

- Insert Beginning
- Insert End
- Insert Position
- Delete Beginning
- Delete End
- Delete Position
- Search
- Traverse

Visualization

10 → 20 → 30 → NULL

Animations

- Node creation
 - Pointer movement
 - Node deletion
 - Search highlighting
-

Phase 6 — Stack Visualizer

Operations

- Push
- Pop
- Peek
- Clear

Visualization

TOP

40
30

20
10

Animations

- Push from top
- Pop animation
- Overflow warning
- Underflow warning

Phase 7 — Queue Visualizer

Operations

- Enqueue
- Dequeue
- Peek
- Reset

Visualization

Front

10 | 20 | 30 | 40

Rear

Animations

- Item movement
- Queue shifting

Phase 8 — Tree Visualizer

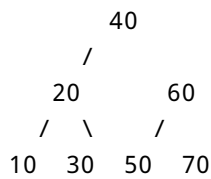
Operations

- Insert
- Delete
- Search

Traversals

- Inorder
- Preorder
- Postorder

Visualization



Animations

- Node insertion
- Traversal highlight
- Search path

Phase 9 — Binary Search Tree

Same UI as Tree

Additional

- BST Validation
- Search Path
- Automatic Positioning

Phase 10 — Heap Visualizer

Types

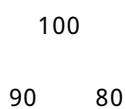
- Min Heap
- Max Heap

Operations

- Insert
- Delete Root
- Heapify

Visualization

Tree + Array representation



60 70 40 20

Array

[100,90,80,60,70,40,20]

Phase 11 — Graph Visualizer

Features

- Add Node
- Delete Node
- Add Edge
- Remove Edge

Algorithms

- BFS
- DFS

Animations

- Node highlighting
- Edge traversal
- Queue visualization

Phase 12 — Searching Algorithms

Linear Search

Controls

- Play
- Pause
- Restart
- Speed

Visualization

Highlight every visited element.

Binary Search

Visualization

Display

- Left
- Right
- Mid

Highlight active region.

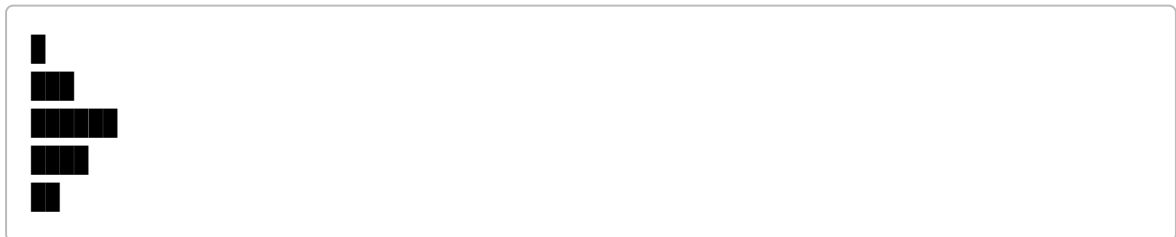
Phase 13 — Sorting Visualizer

Algorithms

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort

Visualization

Bars



Controls

- Play
- Pause
- Step
- Restart
- Speed Slider

Animations

- Compare
 - Swap
 - Sorted section
 - Active element
-

Phase 14 — Learning Panel

Every page should include

Concept

Explain the topic.

Working

Step-by-step explanation.

Real Life Applications

Where is it used?

Advantages

Disadvantages

Complexity Table

Operation	Best	Average	Worst	Space
-----------	------	---------	-------	-------

Phase 15 — Global Controls

Each visualization should support

- Play
 - Pause
 - Restart
 - Next Step
 - Previous Step
 - Reset
 - Speed Slider
-

Phase 16 — User Experience Improvements

Responsive Design

Dark Mode

Light Mode

Fullscreen Mode

Keyboard Shortcuts

Smooth Animations

Loading Skeletons

Toast Notifications

Phase 17 — Open Innovation (Extra Features)

These can help your submission stand out.

AI Features

- AI Explanation
- AI Code Explanation
- AI Interview Questions

Learning Features

- Interactive Quiz
- Progress Tracker
- Achievement Badges
- Daily Challenge

Developer Features

- Code Viewer
- Pseudocode Viewer
- Multiple Programming Languages

Comparison Features

Compare

- Bubble vs Merge
- DFS vs BFS
- Stack vs Queue

Playground

Users can enter custom input.

Share

Generate shareable links for visualizations.

Phase 18 — Testing

Test

- Arrays
- Linked List
- Stack
- Queue
- Trees
- BST
- Heap
- Graph
- Sorting
- Searching

Check

- Mobile
 - Tablet
 - Desktop
-

Phase 19 — Deployment

Deploy on

- Vercel

Verify

- Performance
 - SEO
 - Responsive Design
 - Accessibility
-

Phase 20 — Documentation

Create

README.md

Include

- Project Overview
 - Features
 - Installation
 - Tech Stack
 - Folder Structure
 - Screenshots
 - Live Demo
 - Future Improvements
-

Final Deliverables

- ✓ Source Code
- ✓ Live Website
- ✓ Documentation
- ✓ Setup Guide

(Optional)

- Walkthrough Video
 - Architecture Diagram
 - Feature Proposal
-

Suggested Timeline (10 Days)

Day	Work
1	Project Setup + Landing Page
2	Dashboard + Layout
3	Arrays
4	Linked List + Stack
5	Queue + Trees
6	BST + Heap
7	Graph + BFS + DFS
8	Sorting + Searching
9	AI Features + Polish + Responsive Design

Day	Work
10	Testing + Documentation + Deployment

Final Goal

By the end of the project, users should be able to:

- Learn DSA visually through animations.
- Perform operations on data structures interactively.
- Watch algorithms execute step by step.
- Understand time and space complexity.
- Prepare effectively for coding interviews using a modern, responsive, and educational platform.